
Lab W06: An Adventuring Party, Part 2

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Picking Up From Part 1

In Eclipse, right click `labw05` in the Package Explorer and choose Copy. Right click an empty area of the Package Explorer and choose Paste. When prompted for a new project name, enter `labw06`. Do all of this week's work in `labw06`; leave `labw05` untouched as your Part 1 submission.

Your new `labw06` project starts as an exact copy: the `adventure` package (`Adventurer.java`, `Party.java`, `Quest.java`), the `app` package (`PartyDemo.java`), and the `files` folder (`adventurers.txt`). Last week those three classes each had one constructor and no guards on their setters, and your own `PartyDemo` read the party in from a file and wrote a report to `files/party_report.txt` using `PrintWriter` and `printf`.

This week adds four things to your classes: a second constructor on `Adventurer`, guards on the setters that need them, a couple of constants, and six new methods. Your driver needs a small update too, covered in Section 6. Nothing here involves interfaces either. Every specification below is complete on its own, so you do not need to flip back to last week's assignment to find anything, but each **New this week** note points out exactly what is changing.

2 Learning Objectives

By the end of this part, you should be able to:

1. **Overload** a constructor, and call one constructor from another using `this(...)`.
2. **Guard** a setter against invalid input, and explain the difference between rejecting a bad value and clamping it into range.
3. **Declare** a constant and use it to define a valid range.
4. **Recognize** when a class's constructor can be replaced entirely by field initializers and Java's implicit default constructor.
5. **Debug** a program by comparing its actual output to a known correct answer.

3 Commenting Requirements

The same syllabus requirements from Part 1 still apply, with one addition: this week **Adventurer** and **Quest** each gain constants, and every static constant field must have a comment. See the syllabus's Code Commenting Requirements section for the full detail.

4 Class Specifications

4.1 Adventurer

Fields, and the first constructor, are unchanged from Part 1: **name** and **characterClass** (**String**), and six **int** ability scores.

Add three constants named **MIN_SCORE**, **MAX_SCORE**, and **DEFAULT_SCORE**, holding 1, 20, and 10 respectively. Think through what each one represents, and what that tells you about how it should be declared: is it something that could differ from one **Adventurer** to the next, or one value every **Adventurer** agrees on?

New this week. A second constructor, one that takes only a name and a character class, and defaults all six ability scores to **DEFAULT_SCORE**. Work out the parameter list yourself. Rather than repeating your first constructor's logic, call it from this one using **this(...)**; look into how constructor chaining with **this** works if you have not used it before.

New this week. Guard all six ability score setters: each one must reject a value outside the range **MIN_SCORE** to **MAX_SCORE**, inclusive. If the value is invalid, leave the field unchanged and print a message saying so instead of changing it.

You should not need to touch your constructor to make this work. It already calls the setters, so the guard applies no matter which constructor is used or where else in your program a setter gets called.

New this week. A private helper and six public methods. Write one private helper method that takes a single ability score and returns its modifier, using the rule from Section 5: the score minus 10, divided by 2. The method's name and its parameter's name are up to you.

Then write six public methods, one per ability (**getStrengthModifier()**, **getDexterityModifier()**, and so on through **getCharismaModifier()**), each of which calls your private helper on the matching field and returns the result.

4.2 Party

Fields are unchanged: **partyName** (**String**) and **members** (**ArrayList<Adventurer>**).

New this week. Delete your **Party()** constructor entirely. Give **partyName** and **members** their starting values directly on the field declaration instead:

```
1 private String partyName = "Unnamed Party";
2 private ArrayList<Adventurer> members = new ArrayList<>();
```

With no constructor written anywhere in the class, Java supplies one implicitly, and it runs these field initializers for you. Compile and run your program again after doing this. Nothing about its behavior should change.

addMember, **getMemberCount**, and both **getMember** methods (the one that takes an index and the one that takes a name) are unchanged.

New this week. Guard **setPartyName**: reject a **null** or blank name, leaving the current name in place and printing a message, the same pattern as the setters in **Adventurer**.

4.3 Quest

Fields are unchanged: `questName (String)`, `difficultyRating (int)`, and `rewardGold (int)`. The constructor signature is unchanged: `Quest(String questName, int difficultyRating, int rewardGold)`.

Add a constant named `MAX_DIFFICULTY`, holding 10. The same reasoning from `Adventurer`'s constants applies here.

New this week. Instead of storing `difficultyRating` directly, *clamp* it: write a private helper method that returns 1 if the rating passed in is below 1, returns `MAX_DIFFICULTY` if it is above `MAX_DIFFICULTY`, and otherwise returns the rating unchanged. Call this helper from the constructor before storing the value.

This is a different guarding strategy from `Adventurer`'s setters on purpose: rejecting and clamping are both legitimate ways to guard input, and you should be comfortable with either.

New this week. Guard `setRewardGold`: reject a negative value the same way the setters in `Adventurer` do.

5 The Ability Modifier

In this game, an ability score is not used directly; it is converted to a smaller *modifier* number, using this rule:

modifier equals the score minus 10, divided by 2.

Table 1 gives the official modifier for every score from 1 to 20. Once your six modifier methods are written, run `PartyDemo` (see Section 6) and check every modifier written to `files/party_report.txt` for every adventurer against this table.

Table 1: Official ability score to modifier chart

Score	Mod	Score	Mod	Score	Mod	Score	Mod
1	-5	6	-2	11	+0	16	+3
2	-4	7	-2	12	+1	17	+3
3	-4	8	-1	13	+1	18	+4
4	-3	9	-1	14	+2	19	+4
5	-3	10	+0	15	+2	20	+5

Every modifier written to the report must match this table exactly, for all four adventurers in the data file. If one or two of them do not match, the reference table is not wrong.

Hint: be careful with division.

6 Updating Your Driver

Your `PartyDemo` already reads `files/adventurers.txt` and writes a report to `files/party_report.txt`, from Part 1. This week, update it rather than starting over:

- Add package `app`; as its first line, and import `Adventurer`, `Party`, and `Quest` from the `adventure` package, now that your classes live in a separate package from your driver.

-
- For each adventurer, your report should now show every ability score alongside its modifier, not the raw score by itself. How that looks on the page, a modifier in parentheses next to its score, or something else you come up with, is your decision.

Everything else, reading the file, building the party, writing the quest, closing your `PrintWriter`, already works and does not need to change. Calling one of your new modifier methods from `PartyDemo` is no different from calling `getStrength()` or any other getter you already use there.

7 Sample Output

Once everything is working, `files/party_report.txt` should contain the same information as below, organized similarly. This is the correct, fully fixed output; if yours differs anywhere, that is exactly where to focus your debugging.

The Coppertop Crew has 4 members.

Elandra	Wizard	STR	8 (-1)	DEX	14 (2)	CON	12 (1)	INT	17 (3)	WIS	13 (1)	CHA	10 (0)
Grom	Fighter	STR	17 (3)	DEX	12 (1)	CON	16 (3)	INT	8 (-1)	WIS	10 (0)	CHA	9 (-1)
Fenwick	Rogue	STR	10 (0)	DEX	18 (4)	CON	12 (1)	INT	13 (1)	WIS	14 (2)	CHA	8 (-1)
Brynn	Cleric	STR	13 (1)	DEX	10 (0)	CON	15 (2)	INT	9 (-1)	WIS	17 (3)	CHA	12 (1)

Quest: Clear the Coppertop Mine

Difficulty: 4 out of 10

Reward: 200 gold

8 Suggested Order

1. Add the constants and the second constructor to `Adventurer`.
2. Add guards to `Adventurer`'s six setters.
3. Add the private helper and six modifier methods to `Adventurer`.
4. Refactor `Party`: delete its constructor, add the field initializers, and guard `setPartyName`.
5. Update `Quest`: add the constant, the clamping helper, and the guard on `setRewardGold`.
6. Update your own `PartyDemo`, following Section 6, and run it.
7. Open `files/party_report.txt` and check every printed modifier against Table 1, fixing anything that does not match, using the hint in Section 5 if you need it.

9 Grading

This lab is graded using the checklist below. Each item is graded as met or not met, there is no partial credit within a single item, but each item's point value counts toward your total score.

Table 2: Submission and setup, 1 point each

Item	Points
Submitted as a reasonable attempt	1
Adventurer.java, Party.java, and Quest.java compile and run in Eclipse without errors, in the adventure package	1
PartyDemo.java compiles and runs in Eclipse without errors, in the app package	1
PartyDemo.java declares package app and imports the three adventure classes	1
Project pushed to GitHub, following the walkthrough video	1

Table 3: Code quality

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	2
No unrequested extra code or output	1
Adventurer.java has required documentation, including its constants	2
Party.java has required documentation	2
Quest.java has required documentation, including its constant	2
Loop and method logic reads clearly from top to bottom	1

Table 4: Adventurer

Item	Points
Second constructor sets default ability scores	1
Second constructor calls the first	1
setStrength guards its input	1
setDexterity guards its input	1
setConstitution guards its input	1
setIntelligence guards its input	1
setWisdom guards its input	1
setCharisma guards its input	1
A private helper computes modifiers for all six methods	1
getStrengthModifier returns the correct value	1
getDexterityModifier returns the correct value	1
getConstitutionModifier returns the correct value	1
getIntelligenceModifier returns the correct value	1
getWisdomModifier returns the correct value	1
getCharismaModifier returns the correct value	1

Table 5: Party

Item	Points
No constructor is written anywhere in the class	1
partyName and members are given values through field initializers	1
setPartyName guards its input	1

Table 6: Quest

Item	Points
difficultyRating is clamped using a private helper	1
setRewardGold guards its input	1

Table 7: Reading and reporting

Item	Points
Creates files/party_report.txt	1
Report shows every adventurer's raw score and modifier for all six abilities	4
Report still includes the quest information from Part 1	1
Output formatted correctly	2

Total: 44 points.

10 Submission

Submit your entire `labw06` Eclipse project to your course GitHub repository, per the syllabus's submission guidelines, not just the individual `.java` files. Your repository should reflect the same structure you built in Eclipse: a `src` folder containing the `adventure` package (`Adventurer.java`, `Party.java`, `Quest.java`) and the `app` package (`PartyDemo.java`), plus the `files` folder containing `adventurers.txt`. It is fine either way whether `files/party_report.txt` ends up in your submission or not, since it is generated automatically whenever your program runs.